

職務経歴書

2026年6月22日現在

氏名: 近藤智広（コンドウ トモヒロ）

■職務要約

自社WebサービスおよびSaaS開発において、フロントエンド・バックエンド・インフラを横断したアーキテクチャ設計と実装・運用改善に従事。React / Next.js / Vue.js、Node.js、PHP / Laravel、TypeScriptを軸に、高負荷なバックエンドAPI設計、DBパフォーマンスチューニング、マルチテナントSaaSのインフラ最適化（AWS/GCP/Azure）、Terraformを用いたIaC化を一貫して担当。特に、機密性の高いデータを扱うシステムのセキュリティ設計や、大規模データ処理における非同期アーキテクチャの構築を強みとしている。

直近では、開発が中断していたSaaSの再構築をテックリードとして主導し、コードベースの棚卸しからアーキテクチャ再設計、CI/CD再構築までを実施。また、AI開発支援ツールや生成AI APIを活用した開発生産性向上のワークフロー整備や、複数チームを横断した技術的意思決定のリードも経験。単に仕様を実装するだけでなく、**ビジネス要件と技術的制約のバランス**を考慮した設計と、組織全体の技術力を底上げするレビュー文化の醸成に貢献してきた。

■キャリア成長の概要

2015年～2017年は実装・テスト・不具合修正を中心に基礎を習得。2018年～2021年は機能単位の実装担当、API担当、サブリーダーとしてタスク整理やレビュー補助に関与。2021年～2024年は主担当領域での機能リード、バックエンドリードとして設計方針整理・性能改善・リリース確認を担当。2024年～2025年はテックリードとしてAI活用推進・開発手順標準化・若手メンバーのオンボーディングに加え、準PMとしてプロジェクト全体の進捗管理や関係者調整も担当。2026年以降は契約テックリードとして、開発中断プロダクトの再構築や技術的負債解消を主導している。

■学歴・キャリア開始補足

2015年10月～2016年3月は在学中の長期インターンとして開発補助・テスト補助に従事。2016年3月に大学を卒業後、2016年4月より同社へ正社員として入社し、Webアプリケーション開発を本格的に担当。職務経歴上の実務開始は長期インターン参加時点から記載しているが、正社員としての就業開始は2016年4月である。

■職務経歴

2026年1月～現在 / 株式会社オヤテックジャパン（OyaTech Japan, Inc.）（契約／フルリモート）

開発が中断していたマーケティングオートメーションSaaSの再構築プロジェクトを主導。フロントエンド（React/Next.js）、バックエンド（Node.js/Laravel）、インフラ（AWS/Terraform）を横断し、技術的負債の解消と機能の再設計・再実装を担当。海外エンジニアを含む開発チームの技術指導・品質管理も実施。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【既存SaaS再構築・技術的負債解消】（2026/01～現在） 某スタートアップが開発途中で中断していたマーケティングオートメーションSaaSの再構築を担当。コードベース全体の棚卸しから開始し、技術的負債の解消と機能の再設計・再実装を主導。 【業務内容】 ・コードベース約15万行の棚卸しを実施。依存関係の可視化、未使用コードの特定、廃止予定APIの洗い出しを行い、技術的負債を「即時解消」「段階的解消」「現状維持」の3カテゴリに分類。 ・Next.js / TypeScript を用いたフロントエンド再構築では、コンポーネント設計を見直し、共通化可能なUI部品をAtomic Designベースで再定義。状態管理はRedux ToolkitからZustandへ移行し、不要な再レンダリングを削減。 ・Laravel APIのリファクタリングでは、コントローラの肥大化を解消するためService-Repositoryパターンを導入。新機能APIの設計ではOpenAPI（Swagger）を先行定義し、フロントエンドとの開発並行を可能にした。 ・Terraformを用いたAWSインフラ再設計では、従来の手動EC2運用からECS Fargateへ移行。RDSはスナップショットからのリストアを自動化し、S3とCloudFrontを組み合わせた静的アセット配信設計を実施。 ・GitHub ActionsによるCI/CDパイプライン再構築では、Lint、単体テスト（Jest/PHPUnit）、セキュリティスキャン（Trivy）、デプロイを直列化。環境変数はSecrets Managerと連携し、ステージング/本番で切り替わる仕組みを構築。 ・海外エンジニア（欧州・アジア在住）3名のオンボーディングを実施。非同期コミュニケーションを前提としたPRレビューガイドラインを作成し、週次の非同期ステータスレポート運用を開始。 【課題・工夫】 最大の課題は、要件定義書や設計書がほぼ存在しない状態での再構築だった。コードリーディングと実際の画面操作を並行して実施し、想定ユースケースを逆推測した。決定した設計方針はすべてADR（Architecture Decision Record）として英語でドキュメント化し、海外メンバーと認識合わせを行った。 インフラは既存の手動運用スクリプトが散在していたため、まずステージング環境をTerraformで先行構築し、本番との差分を都度検証するサイクルを構築。DBマイグレーションはロールバック計画をセットで設計し、データ損失リスクを最小化した。	【言語】 TypeScript 5.7, PHP 8.3 【FW】 Next.js, Laravel, Node.js 【インフラ】 AWS (ECS, RDS, S3, CloudFront) 【その他】 Terraform, GitHub Actions, Docker	【規模】 5名（海外3名含む） 【役割】 契約テックリード 【従事割合】 約80%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
中断前のチームが残した未完了タスクが数百件に上ったため、優先度を「ユーザー影響度」「実装工数」「依存関係」の3軸でスコアリングし、実行計画を策定。 【実績】 ・約3ヶ月でプロダクトを再稼働可能な状態に復旧。主要機能（キャンペーン作成、メール配信、分析ダッシュボード）を利用可能に。 ・インフラ構築時間を手動の数日からTerraform実行の約30分に短縮。環境再現性が向上し、新規開発環境のセットアップも半日から1時間以内に改善。 ・CI/CD整備によりデプロイ頻度を週1回から週3〜4回に向上。デプロイ失敗率も従来の約15%から2%未満に低減。 ・海外エンジニア3名のオンボーディングを完了し、現在はチーム全体で週あたり8〜10件のPRを安定してマージできる状態。		

【今後の志向】 高セキュリティ・高信頼性が求められるSaaSやプラットフォーム開発において、アーキテクチャ設計から運用改善までをリードし、事業成長に直結する技術貢献を続けたいと考えている。

2024年4月～2025年12月 / 株式会社SKIウエスト（フルリモート）

マーケティング支援SaaS、サイト売買プラットフォーム、ホスティング管理機能など複数の自社サービス開発に従事。主担当領域ではフルスタックエンジニア兼テックリードとして、React / Next.js、Node.js、Laravel、AWSを用いた機能開発・運用改善・AI活用推進を担当。加えて準PMとして開発スケジュール管理・工数調整・ステークホルダーへの進捗報告・リスク管理も担当。新規開発と既存機能の保守・問い合わせ対応が並行する体制で、技術方針整理・PRレビュー・開発手順の標準化・リリース後の改善や引き継ぎも継続して担当。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【ホスティング管理・AI機能連携】（2025/07～2025/12） ホスティング管理画面とサポート向けAI検索機能の改善を担当。ドメイン設定・SSL状態・DNS反映状況・バックアップ確認など、ユーザーが問い合わせしやすい領域を整理し、画面上で自己解決できる導線とサポート担当者向けの調査補助機能を開発。 【業務内容】 ・サポートチケット約500件を分析し、問い合わせ頻度が高い項目（DNS設定ミス、SSL期限切れ、バックアップリストア手順）を特定。ユーザー向け画面では「現在の状態」「次取るべき操作」「想定反映時間」をひと目で把握できるタイムラインコンポーネントをNext.jsで実装。 ・Laravel API側では、各契約のDNSレコード状態を外部サービス（AWS Route53、Cloudflare API）と連携してリアルタイム取得する処理を実装。SSL証明書の有効期限はLet's EncryptのAPIから取得し、期限切れ30日前から画面上で警告表示する機能を追加。 ・OpenAI API（GPT-4）を用いたサポート向けAI検索機能では、社内FAQ・マニュアル・過去の問い合わせ履歴約2,000件をベクター化し、RAG（検索拡張生成）を構築。ユーザーの質問に対して回答文＋参照元リンク＋確認すべき管理画面項目をセットで表示する仕様を設計。 ・Terraformを用いてCloudFrontのキャッシュ挙動とWAFのルールセットを変更し、管理画面への不正アクセス対策を強化。ステージング環境でDNS状態ごとの表示を網羅的に検証するためのE2Eテスト（Playwright）を整備。 ・サポート担当者向けに、AI出力の信頼性を確保するための確認観点リストを作成し、運用手順書に反映。 【課題・工夫】 DNS反映はプロバイダやTTL設定によって反映時間が大きく異なるため、一律のメッセージ表示ではユーザーに不安を与えていた。経過時間に応じて「通常は数分～数時間」「24時間以上経過している場合はプロバイダへ問い合わせ推奨」といった段階的なガイダンスを導入。 AI回答については、過去の古いFAQを参照して誤った手順を提案するケースが見られたため、参照元の更新日時を回答に必ず付与し、3ヶ月以上更新がない情報は検索対象から除外するフィルタを実装。また、AI回答はあくまで「候補」として位置付け、最終判断はサポート担当者が行うフローを徹底した。性能面では、ベクター検索にPineconeを採用し、クエリ応答時間を平均200ms程度に収めるよう調整。OpenAI APIの利用コストを抑えるため、類似質問が過去にあればキャッシュから返す仕組みも併せて実装。 【実績】 DNS / SSL関連の月次問い合わせはリリース前後の比較で約18%減少（月平均42件→34件）。サポート担当者の初動調査時間は1件あたり平均15分程度短縮（平均25分→10分）。AI回答は約70%のケースで担当者の一次調査として活用され、エスカレーション件数も約20%減少。	【言語】 TypeScript 5.7, PHP 8.3 【FW】 Next.js, Laravel 【インフラ】 AWS (CloudFront, ECS, RDS, WAF) 【その他】 Terraform, GitHub Actions, OpenAI API, Pinecone	【規模】 10名 【役割】 テックリード / 準PM / フルスタックエンジニア 【従事割合】 約65%
【サイト売買プラットフォーム改善】（2025/04～2025/12） 売り手・買い手の交渉・本人確認・契約・エスクロー入出金に関わる管理機能と通知機能の改善を担当。サービス運営メンバーが案件状態を素早く判断できるよう、審査・契約・入金・譲渡状況を一画面で追跡できる管理導線を整備。 【業務内容】 ・Laravel / TypeScript による案件管理・本人確認・審査画面の改修。状態遷移を「交渉中→本人確認中→契約確認中→入金待ち→譲渡確認中→完了」の6段階で定義し、各ステータスに応じたアクションボタンと確認項目を動的に表示。 ・Node.js（Express）でWebhookエンドポイントを実装し、決済代行サービス（Stripe連携）からの入金通知を受け取って契約ステータスを自動更新。再送処理と冪等性を確保するため、イベントIDをキーにした重複チェックを実装。	【言語】 PHP 8.2, TypeScript 5.6 【FW】 Laravel, Vue.js, Node.js 【監視】 Sentry, CloudWatch 【その他】 Docker, AWS	【規模】 7名 【役割】 機能リード / フルスタックエンジニア 【従事割合】 約60%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
・通知機能では、メールテンプレートをTwigで管理し、契約ステータス変更ごとに送信するタイミングと宛先を定義。送信履歴と再送機能を管理画面に追加。 ・SentryとCloudWatch Logsを連携し、決済Webhookのエラー（署名不一致、タイムアウト）をリアルタイムで検知。エラー発生時はSlack通知が飛ぶようにし、一次対応の属人化を防止。 ・運営メンバー向けに、操作履歴と監査ログを別テーブルで保持し、誰がいつどのステータスを変更したかを追跡できる仕組みを導入。 【課題・工夫】 案件状態が複数に分かれ、各画面で表示が分散していたため、運営メンバーが「次に何をすべきか」を判断しづらかった。状態遷移図をMiroで作成し、チームで合意した上で一画面化を実現。特に「入金待ち」と「譲渡確認中」はユーザー側のアクション待ちとなるため、最終更新日時からの経過時間を表示し、放置案件を早期に発見できるようにした。 決済Webhookは外部サービスからの呼び出しのため、タイムアウトやネットワーク障害を考慮し、SQSを用いた非同期処理に変更。失敗時はリトライキューに再投入し、3回失敗したら運営へ手動対応を促すアラートを送信する設計とした。 【実績】 案件確認にかかる時間を1件あたり平均8〜10分から5分前後に短縮。ユーザーからの「次に何をすべきか分からない」という問い合わせも対象カテゴリで約20%減少。Webhookエラーによる決済遅延は月平均3件からほぼ0件に改善。	(ECS, SQS)	
【キーワード調査SaaS機能改善】（2024/10〜2025/06） 検索データを活用するマーケティング支援SaaSにて、調査結果画面・保存リスト・CSV出力・API連携機能の改善を担当。大量データ表示時の体感速度・絞り込み操作・外部ツール連携の使いやすさを改善するため、画面・API・DBの各層を横断して対応。 【業務内容】 ・Next.js / TypeScript による検索結果・保存リスト・CSV出力画面の改善。検索結果は仮想スクロール（react-window）を導入し、一度にレンダリングする行数を制限して描画パフォーマンスを向上。 ・Node.js APIのページングをoffset/limitからcursorベースに変更し、大量データ取得時の負荷を分散。フィルタ条件はクエリパラメータで受け取り、動的にSQLを生成する仕組みを実装。 ・Aurora MySQLのスロークエリログを分析し、インデックスが効いていないカラムに複合インデックスを追加。Redisには検索結果のキャッシュを導入し、同一条件のリクエストはDBアクセスをスキップ。 ・GitHub Actionsに、マイグレーションのロールバックテストと、負荷テスト（k6）のステップを追加し、デプロイ前に性能劣化を検出できるパイプラインに拡張。 ・PdM・デザイナー・CS担当との定例会議を週次で開催し、問い合わせ内容とユーザー行動ログをもとに改善優先度を決定。API利用者向けにエラーコード表とレート制限の説明をDeveloper Portalに掲載。 【課題・工夫】 検索条件が増えるにつれて、単一のSQLクエリが複雑化し、実行計画が不安定になる問題があった。クエリビルダを段階的に構築し、条件の有無でJOINやWHERE句を動的に切り替える実装に変更。また、CSV出力は大量データ（最大10万件）のため、同期処理ではタイムアウトが頻発していたため、バックグラウンドジョブ（AWS Batch）で生成し、完了後にS3からダウンロードする非同期フローに全面改修。 APIのレスポンス形式はフロントエンドの表示項目に合わせて過剰にネストしていたため、必要なフィールドのみを返す「Fields Selector」パターンを導入し、ペイロードサイズを平均40%削減。 【実績】 保存リストとCSV出力の導線を整理し定型的な調査作業の操作数を平均6クリックから3〜4クリックへ削減。大量データ検索時の主要API応答時間は平均1.8秒前後から1.1秒前後へ改善。CSV出力は非同期化によりタイムアウト発生を月数件からほぼ0件に抑制。	【言語】 TypeScript 5.5 【FW】 Next.js, Node.js 【DB】 Aurora MySQL, Redis, BigQuery 【その他】 Docker, GitHub Actions, AWS CodePipeline	【規模】 13名 【役割】 機能リード / フルスタックエンジニア 【従事割合】 約60%
【AI活用ワークフロー整備】（2024/04〜2024/09） 開発支援AIを活用したIssue整理・実装補助・レビュー観点作成のワークフローを整備。既存のGitHub運用にAI活用を組み込み、機密情報の扱い・出力確認・レビュー責任を明確にした上でチームで再利用できるテンプレートを作成。 【業務内容】 ・GitHub Copilot / Claude / Clineを用いた実装補助・差分説明・レビュー補助の検証を、実際の小規模バグ修正タスクで実施。出力品質を比較し、用途別に最適なツールを選定。 ・Issue分解テンプレート、PR説明テンプレート、テスト観点作成テンプレート、リファクタリング方針比較テンプレートの4種類をMarkdownで作成。テンプレートには「AIが生成した部分」「人が確認すべき部分」「判断保留部分」を明示するセクションを設けた。 ・AI利用時の機密情報（APIキー、個人情報）の取り扱いポリシーを策定。社内ルールとして「AIに貼り付けてよい情報／禁止情報」をリスト化し、セキュリティチェックリストとして共有。 ・チーム向け勉強会（全3回）を実施。第1回はAIツールの基本操作、第2回はプロンプトエンジニアリングの実践、第3回は生成コードのレビュー観点と脆弱性チェックをテーマに開催。オンボーディング資料にAI活用章を追加。 【課題・工夫】 AIの出力は常に正しいとは限らず、特に古いライブラリのコードを生成するケースで非推奨APIを提案することがあった。生成コードを採用する前に必ず公式ドキュメントと照合するステップをレビュー項目に追加。機密情報の誤送信を防ぐため、GitHub Copilotの「禁止ファイルパターン」設定を活用し、.envやconfig/secrets配下のファイルを補完対象から除外した。 チームメンバーからは「AIに依存しすぎないか」という懸念があったため、あくまで「補助ツール」として位置づけ、最終的な設計判断とコード品質の責任は人間が持つことを明文化。	【言語】 TypeScript 5.4, Markdown 【AI】 Claude, Cline, GitHub Copilot 【管理】 GitHub, Backlog 【その他】 Slack, Google Meet	【規模】 3名 【役割】 AI活用推進担当 / 開発フロー整備 【従事割合】 約30%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【実績】 定型的な実装準備・PR説明・レビュー観点作成の品質を標準化。小規模IssueではPR説明・テスト観点作成にかかる時間を平均30〜40%短縮。レビュー差し戻しも導入前と比較して約15%減少。チーム内のAI利用率は約70%に達し、自主的にテンプレートを拡充するメンバーも出てきた。		

【退職理由】 自社Webサービス開発の経験を活かし、より多様なプロダクトの再構築や技術的負債解消に挑戦するため、契約ベースのテックリード案件へ移行。

2021年4月〜2024年3月 / 株式会社Phoenixテクノロジーズ

BtoB SaaS・予約管理・会員管理・モバイルAPIなどのWebサービス開発に従事。React / Vue.js、Node.js、Laravel、AWSを用いた機能追加・性能改善・運用改善を担当。中盤以降は主担当領域での機能リードやレビュー担当として設計方針整理・コードレビュー・リリース確認・若手メンバーの実装相談に加え、準PMとして開発スケジュール統括や顧客調整も担当。主担当案件とは別に既存SaaSの保守・障害調査・問い合わせ対応を継続的に実施。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【モバイルAPI・CI/CD整備】（2024/02〜2024/03） モバイルアプリ向けAPIとCI/CD整備に参画。短期間でAPI仕様・モック・認証・通知・デプロイ手順を整備する必要があり、モバイルチームと並行開発できるようAPI契約と検証環境を優先して構築。 【業務内容】 ・ NestJS / Node.js を用いたAPIゲートウェイを設計。認証はJWT（Access Token + Refresh Token）方式を採用し、エラーレスポンスはRFC 7807（Problem Details）に準拠した統一形式を定義。 ・ Flutter（Dart）チームと協力し、OpenAPI 3.0仕様書を先行作成。Swagger EditorでAPI設計を可視化し、モックサーバー（Prism）を立ち上げることで、バックエンド実装完了前にモバイル側の開発を進められる体制を構築。 ・ AWS ECS / ALBを用いた検証環境を構築。ECSタスク定義にはFargateを採用し、オートスケーリングはCPU使用率70%を閾値に設定。ALBのヘルスチェックは/healthエンドポイントで実施。 ・ GitHub ActionsでLint（ESLint）、単体テスト（Jest）、Dockerビルド、ECSデプロイまでのパイプラインを構築。ステージング環境へのデプロイはmainブランチへのマージをトリガーとし、本番環境は手動承認ステップを挟むフローを実装。 【課題・工夫】 プロジェクト期間が2ヶ月と短く、モバイルチームとAPIチームで開発スケジュールが異なるため、相互にブロックしない仕組みが必須だった。OpenAPI仕様を最優先で固め、APIの入出力インターフェースを確定させた。モックサーバーにより、モバイル側はダミーデータでUI実装を進められ、結合テスト時に初めて問題が発覚するリスクを低減。 認証フローはセキュリティ要件が厳しかったため、リフレッシュトークンはHTTP Only Cookieで保持し、Access Tokenはメモリに保持する二重管理方式を採用。モバイル側にシークレットを一切持たせない設計とすることで、漏洩リスクを抑えた。 【実績】 API仕様起因の手戻りは結合テスト期間中に約30%減少。検証環境への反映作業も手動30分程度からGitHub Actions経由で10分以内に短縮。モバイルチームからは「モックがあるおかげでUI実装がスムーズに進んだ」とのフィードバックを得た。	【言語】 TypeScript 5.3, Dart 3.2 【FW】 NestJS, Flutter 【インフラ】 AWS (ECS, ALB, S3) 【CI/CD】 GitHub Actions, Terraform 【その他】 Docker, Swagger	【規模】 8名 【役割】 API設計・ CI/CD担当 【従事割合】 約35%
【SaaS業務管理刷新】（2023/02〜2024/01） SaaS型業務管理システムのリニューアルPJにてバックエンド設計・実装を担当。既存機能の仕様がドキュメント化されていない箇所が多かったため、現行画面・DB・運用手順を確認しながらLaravel APIとReact画面へ段階的に移行。 【業務内容】 ・ PHP / Laravel を用いた新API設計。認証はLaravel Sanctumを採用し、権限管理はSpatie/laravel-permissionパッケージをベースに、カスタムロール（管理者・一般ユーザー・閲覧専用）を定義。 ・ OpenAPI（Swagger）ドキュメントを整備し、フロントエンド（React）との仕様連携を効率化。APIレスポンスのフィールド名やエラーコードは既存システムと互換性を保つよう調整。 ・ React SPAではコンポーネントをAtomic Designで分割。フォーム状態管理はReact Hook Form + Zodでバリデーションを実装。エラー表示は一貫性を持たせるため、共通のToast通知コンポーネントを用意。 ・ Redisキャッシュ戦略を構築。一覧・検索系APIはキャッシュ有効期限を5分に設定し、更新系API実行時に該当キャッシュをパージする仕組みを実装。 ・ コードレビューでは、移行差分リスト（約80項目）を活用し、旧仕様との乖離をチェックする観点をチームで共有。週次の共有会で実装方針のすり合わせを実施。 【課題・工夫】 既存システムは約5年間運用され、画面の挙動とDBの状態が必ずしも一致していないケースがあった。移行前にDBの不整合パターンを洗い出し、クレンジングバッチを事前実行した。 新旧システムの並行稼働期間中は、両方のDBに同じデータを書き込む必要があったため、Laravelのイベントリスナーを活用し、旧システムのAPIを非同期で呼び出す双方向同期処理を実装。同期失敗時のリトライ機構もRedisキューで実現。 移行差分リストはExcel管理ではなく、GitHub Issuesで管理し、進捗状況を可視化。完了した項目は自動的にチェックされるワークフローを構築。 【実績】 既存仕様差分を約80項目一覧化し移行時の認識違いによる手戻りを削減。検索・一覧系APIの平均レス	【言語】 PHP 8.1, TypeScript 5.0 【FW】 Laravel, React 【DB】 MySQL, Redis 【その他】 Docker, GitHub Actions, Swagger	【規模】 9名 【役割】 バックエンド リード / 準PM 【従事割合】 約70%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
ポンスは約1.5秒から0.9秒前後へ改善。並行稼働期間中のデータ不整合は月平均2件まで抑制。		
【予約・会員管理システム機能拡張】（2021/11～2023/01） 予約管理・会員管理システムの機能拡張と性能改善を担当。予約枠・会員ステータス・通知履歴・夜間集計バッチの処理が複雑化していたため、運用上の確認手順とシステム処理の両面から改善を実施。 【業務内容】 ・Node.js / TypeScript を用いた予約API（予約作成・変更・キャンセル）の改修。排他制御を考慮し、予約枠の二重予約を防ぐため、DBの楽観ロック（バージョンカラム）を導入。 ・通知API（メール・SMS）はキュー駆動（BullMQ + Redis）に変更し、大量通知時の負荷を分散。バッチ処理（夜間集計・会員ステータス更新）は、従来の単一プロセスから、並列処理（Worker単位で分担）に変更。 ・MySQLのスロークエリログを分析し、予約一覧検索で実行されているN+1クエリを発見。Eloquentのwith句を用いたEagerロードに修正し、インデックスが不足していたカラムに複合インデックスを追加。 ・Vue.js管理画面では、検索条件に日付範囲・会員種別・予約ステータスを追加し、リアルタイム更新機能（WebSocket）で新規予約やキャンセルを即座に反映。 ・JMeterを用いた負荷テストを実施し、ピーク時（月末・月初）の想定リクエスト数に対してエラー率1%未満を目標に性能調整を実施。 【課題・工夫】 夜間バッチ処理は、予約データの集計、会員ランク更新、未処理通知の再送など複数のジョブが直列で実行されており、全体の処理時間が約95分かかっていた。各ジョブの依存関係を分析し、並列実行可能なジョブを分離。Node.jsのWorker Threadsを利用して並列化した。 バッチ処理中にDBロックが発生し、日中の予約操作に影響を与えるケースがあったため、バッチのトランザクションを分割し、1回のコミットで処理する件数を1000件単位に制限。ロック時間を大幅に短縮した。 運用担当者からは「問い合わせ時に複数画面を行き来するのが負担」という声があったため、予約詳細画面に会員情報・通知履歴・操作ログをタブで切り替えられるダッシュボード形式に統合。 【実績】 夜間バッチの処理時間を約95分から約55分へ短縮。予約枠・会員状態・通知履歴を同一画面で確認できるようにしたことで運用担当者の問い合わせ確認時間を1件あたり約10分削減。負荷テストではピーク想定アクセス時のエラー率0.3%を達成。	【言語】 TypeScript 4.5 【FW】 Node.js, Vue.js 【DB】 MySQL 【テスト】 JMeter, xUnit 【その他】 Azure DevOps, Git	【規模】 7名 【役割】 機能リード 【従事割合】 約55%
【BtoB SaaS保守運用・改善】（2021/04～2021/10） 既存BtoB SaaSの保守運用と機能改善を担当。障害調査・問い合わせ対応・管理画面改修・データ修正依頼への対応を通じて既存サービスの構造と運用課題を把握。 【業務内容】 ・障害発生時の一次切り分けを担当。Sentryで通知されたエラーを分類し、頻発エラーの原因を調査。一時対応としてリトライ処理やタイムアウト値の調整を実施。 ・CSからの問い合わせに対し、調査用SQLを作成。複雑な結合が必要な場合はビューを作成し、運用チームが再利用できるように整備。 ・Laravel管理画面の検索条件拡充やCSV出力項目の追加を対応。既存画面のUI微修正も継続的に実施。 ・CloudWatch Logsで特定エラーパターンの発生件数を可視化するダッシュボードを作成し、週次の運用定例会で共有。 【課題・工夫】 Sentryの通知数が多く、重要なエラーが埋もれる状態だった。エラーレベルを再定義し、FatalとErrorだけSlack通知するようにフィルタリング。エラー発生時のコンテキストをSentryに必ず送信するよう実装を統一し、調査効率を向上させた。 問い合わせ対応では、同様の質問が繰り返されるケースが多かったため、調査用SQL集を作成し、社内Wikiに掲載。これにより、新メンバーでも一定レベルの調査ができるようになった。 データ修正依頼は本番DB直操作を避けるため、専用のArtisanコマンドを作成し、実行前にドライランで影響範囲を確認できる仕組みを導入。 【実績】 障害調査時に確認すべきログ・画面・SQLを整理し一次調査の属人化を軽減。定型問い合わせの一次調査時間は平均20分前後から10～12分程度へ短縮。Sentryの重要エラー検出漏れはゼロになった。	【言語】 PHP 7.4, SQL (MySQL 5.7) 【FW】 Laravel 【DB】 MySQL 【監視】 Sentry, CloudWatch 【その他】 GitHub, Backlog	【規模】 5名 【役割】 メンバー 【従事割合】 約50%

【退職理由】 受託・準委任型の開発から、よりユーザーの反応を見ながら継続改善できる自社サービス開発へ軸足を移したいと考え退職。

2015年10月～2021年3月 / 株式会社情報技術

2015年10月～2016年3月は在学中の長期インターンとして参画し、2016年4月より正社員として入社。初期は先輩エンジニアの指示のもとテスト・軽微な改修・ドキュメント整備・障害票調査を担当。その後ECサイト・CMS・業務管理画面・モバイルAPI・AWS移行などを経験しWebアプリケーション開発の基礎を段階的に習得。後半は小規模タスクの取りまとめ・レビュー補助・リリース確認も担当。

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
【EC・CMS基盤AWS移行】（2019/11～2021/03） ECサイトとCMS基盤のAWS移行PJに参加。オンプレミス環境で稼働していたLaravelアプリケーションをAWS ECS / RDS / S3 / CloudFront構成へ移行するため、アプリ改修・データ移行・検証・本番切替支援を担当。 【業務内容】	【言語】 PHP 7.4, Python 3.8 【FW】 Laravel	【規模】 9名 【役割】 サブリーダー 【従事割合】

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
・ PHP / Laravel アプリケーションの環境依存処理を改修（ファイル保存パスをローカルからS3へ変更、メール送信設定をSMTPからSESへ切り替え）。 ・ AWS ECS / RDS / S3 / CloudFront 構成での動作検証。ECSタスク定義ではFargateを選択し、RDSはMulti-AZ構成で高可用性を確保。 ・ GitHub ActionsによるCI/CDパイプラインを整備。mainブランチへのプッシュをトリガーに、テスト（PHPUnit）→Dockerビルド→ECSデプロイを自動化。 ・ データ移行パッチ（Python）を開発。オンプレのMySQLダンプを取得し、RDSへリストア。移行前後の件数チェックと整合性検証を実装。 ・ 画像・添付ファイル（約5万ファイル）のS3移行。パス変換処理とCloudFront経由の配信確認を実施。移行後も旧URLでのアクセスをリダイレクトするApacheのRewriteルールをCloudFront Functionsで再現。 ・ 小規模タスクの進捗確認とレビュー補助、本番切替当日の確認項目リスト（約60項目）を作成して運用チームと共有。 【課題・工夫】 コード内にハードコードされた絶対パスが多数存在し、移行後にリンク切れが発生するリスクがあった。移行前に全てのファイル参照箇所を洗い出し、環境変数で動的に切り替えられるよう改修。画像URLの変換はPythonスクリプトで一括処理し、旧URL→新URLのマッピングテーブルを作成。データ移行中にサイトを停止できないという要件があったため、移行パッチはバックグラウンドで実行し、最終的な差分のみをダウンタイム中に適用する戦略を採用。本番切替は深夜帯に実施し、ロールバック手順も事前に複数回シミュレーションして問題を洗い出した。 移行後の性能検証では、CloudFrontのキャッシュヒット率が当初60%程度だったため、Cache-Controlヘッダーを調整し、静的アセットのキャッシュ期間を1年に延長。結果的にヒット率は85%まで向上。 【実績】 移行手順書・ロールバック手順・切替時の確認項目を整備し、約12万件のデータ移行と画像・添付ファイル移行を大きな障害なく完了。デプロイ手順整備によりリリース作業時間は約40分短縮。CloudFrontキャッシュヒット率の改善により、ページ読み込み速度も約20%向上。	【クラウド】 AWS (ECS, RDS, S3, CloudFront) 【その他】 Docker, GitHub Actions	約70%
【モバイルアプリ新規開発】（2018/10～2019/01） モバイルアプリ（iOS/Android）の新規開発PJにフロント・API双方で参画。React Nativeによる画面実装とNode.js / ExpressによるBFF実装を担当し、認証・通知・アプリ審査対応まで経験。 【業務内容】 ・ React Native（Expo）を用いてログイン画面、一覧画面、詳細画面、設定画面を実装。ナビゲーションはReact Navigationを採用し、認証状態に応じて画面遷移を制御。 ・ Node.js / ExpressによるBFFを設計。既存のREST APIをラップし、モバイル向けにレスポンスを整形（不要フィールドの除外、日付フォーマットの変換）。 ・ Firebase AuthenticationとFirebase Cloud Messagingを導入。プッシュ通知のペイロード設計と、バックグラウンド・フォアグラウンドでのハンドリングを実装。 ・ クラッシュレポート（Firebase Crashlytics）を設定し、リリース後のクラッシュを監視。端末差分による表示崩れを修正。 ・ App Store / Google Play リリース対応。ストアの審査ガイドラインに従い、プライバシーポリシーのリンク設定、スクリーンショットの作成、アプリの説明文を作成。 【課題・工夫】 iOSとAndroidでUIの見た目が微妙に異なる問題に対処するため、プラットフォームごとにスタイルを分岐させるConditional Styleを導入。日付ピッカーやモーダル動作差はクロスプラットフォームライブラリを採用して統一感を出した。 通信エラー時のユーザー体験を改善するため、オフライン時はAsyncStorageに保存されたキャッシュデータを表示し、オンライン復帰時に自動で最新データを取得する戦略を実装。認証トークンの有効期限が切れた場合は、リフレッシュトークンで自動再取得し、ユーザーに再ログインを強制しない仕組みを組み込んだ。 ストア審査では、プッシュ通知の許可ダイアログ表示タイミングが指摘されたため、初回起動時ではなく、実際に通知が必要な機能に表示タイミングを変更し、審査を通過。 【実績】 初回リリース後のクラッシュログと問い合わせ内容をもとに改善を継続。主要クラッシュはリリース後1ヶ月で6件から1件まで減少。通知導線と初回設定画面の改善により初回設定完了率は約12ポイント向上。App Store/Google Playともに4.5以上の評価を獲得。	【言語】 JavaScript ES2019, TypeScript 3.3 【FW】 React Native, Express 【DB】 MySQL, Firebase 【その他】 Docker, Git, CircleCI	【規模】 6名 【役割】 メンバー / API 担当 【従事割合】 約50%
【業務管理SaaS開発】（2017/07～2019/02） 小規模事業者向け業務管理SaaSの開発に参画。申請・承認フロー・通知・権限管理・帳票出力をLaravelで実装しVue.js管理画面と連携することで紙・Excel中心だった業務のWeb化を支援。 【業務内容】 ・ PHP / Laravel で申請フォーム、多段階承認フロー、差戻し処理、承認履歴機能を実装。承認ルートは部署・役職・申請金額によって動的に分岐する仕様だったため、DBでルート定義テーブルを持ち、リレーションで次の承認者を取得する設計を採用。 ・ Vue.js（Vuex使用）で管理画面を実装。申請一覧、検索・絞り込み機能、承認ボタン、コメント入力欄を実装。リアルタイムなステータス更新のため、ポーリング（5秒間隔）で最新情報を取得。 ・ MySQLテーブル設計補助。申請データ、承認履歴、権限設定、部署マスタのスキーマを設計。権限管理は「承認者」「申請者」「管理者」の3ロールに加え、代理承認機能も実装。 ・ Laravel-Excelを用いて申請データのExcel出力機能を実装。帳票テンプレートはBladeで作成し、PDF出力（dompdf）も対応。 ・ 単体テスト（PHPUnit）と結合テストを実施。顧客受入テストでは、実際の業務フローに沿ったシナ	【言語】 PHP 7.1, TypeScript 2.6 【FW】 Laravel, Vue.js 【DB】 MySQL 【その他】 Docker, GitHub Actions	【規模】 6名 【役割】 メンバー / 機能担当 【従事割合】 約60%

プロジェクト詳細（期間・役割）	開発環境	開発規模・役割
リオテストを実施し、不具合修正に対応。 【課題・工夫】 承認ルートが複雑で、条件によって遷移先が変わるため、コードにベタ書きすると保守性が悪化すると判断。承認ルート定義をJSONで保持し、ルールエンジンで動的に評価する仕組みを導入。これにより、新たな承認条件の追加が設定変更だけで済むようになった。 帳票出力では、顧客ごとに出力フォーマットが異なるという要望があったため、Bladeテンプレートを動的に切り替えられる機構を実装。顧客IDに応じて使用するテンプレートファイルを変更し、カスタマイズ性を確保。 申請者が誤った入力をした場合の差戻しフローでは、承認者からのコメントと共に申請を差し戻し、申請者が修正後に再申請できる導線を整備。差戻し理由のプルダウン選択肢をマスタ管理し、運用側で自由に追加・削除できるようにした。 【実績】 承認履歴と通知状態を画面上で確認できるようにし運用担当者の問い合わせ対応を支援。月次の確認作業を約10時間削減。帳票出力のフォーマット差分も吸収し既存業務からの移行後の差戻し問い合わせを約30%削減。顧客満足度調査では「業務効率が大幅に改善した」との評価を得た。		
【中規模ECリニューアル】（2016/02～2017/06） ECサイトのリニューアルPJに参画。商品一覧・詳細・カート・決済・在庫管理機能を対象にフロントエンド改修とLaravel API開発を担当し既存サイトからの段階的な移行を支援。 【業務内容】 ・Reactによる商品一覧、検索、カート画面のフロントエンド改修補助。UIライブラリはMaterial-UIを採用し、レスポンシブデザインを実装。 ・Laravelによる決済API、在庫APIのテストおよび軽微な改修。決済は外部ゲートウェイとの連携が含まれるため、テスト用のモックAPIも作成。 ・MySQLクエリ最適化補助。商品検索で遅延が発生していたため、EXPLAINを実行してインデックス不足を特定。カテゴリIDと商品名に全文検索インデックスを追加。 ・既存画面との仕様差分確認。商品画像の表示サイズ、カートの数量制限、決済後のリダイレクト先など、細かな仕様差異をチェックリストで管理。 ・決済失敗、在庫不足、同時購入、注文キャンセル時の例外処理実装補助。各エラーケースに対応するエラーメッセージとリカバリー導線を実装。 ・デザイナー、バックエンド担当との画面仕様・API仕様調整補助。Figmaでのデザイン確認と、APIレスポンスのフィールド定義を調整。 【課題・工夫】 旧サイトから新サイトへの段階的移行中、カート情報をセッションで管理していたが、旧サイトと新サイトでセッションが共有されず、移行時期にカートがリセットされる問題が発生。ユーザーIDベースでカート情報をDBに保存する方式に変更し、新旧サイト間でカートを引き継げるように改修した。 在庫管理はリアルタイム性が重要だったため、在庫確認APIはRedisでキャッシュし、注文確定時にDBの在庫数をデクリメントする二重チェック方式を採用。同時購入時の樂觀ロックで過剰販売を防止した。決済エラー時のユーザー体験を改善するため、決済失敗時にはエラーコードに応じた具体的な対処法を画面に表示する実装を追加。 【実績】 商品一覧とカート周辺の表示速度改善・既存不具合の修正に貢献（表示時間は約2.4秒から1.6秒前後に改善）。決済・在庫まわりの例外処理整理により購入途中のエラー問い合わせを約15%削減。	【言語】 PHP 7.0, TypeScript 2.3 【FW】 Laravel, React 【DB】 MySQL 【その他】 Docker, Git	【規模】 7名 【役割】 メンバー（実装・テスト補助） 【従事割合】 約70%
【社内業務システム開発補助】（2015/10～2016/01） 社内業務管理システムの開発補助・テスト補助を担当。先輩エンジニアの指導のもと帳票出力の軽微な修正・入力チェック修正・不具合調査・テストデータ作成を通じて実務開発の基礎を習得。 【業務内容】 ・Java / Spring Framework で構築された既存システムの帳票出力機能の軽微な修正（出力項目の追加、列幅調整）。 ・入力チェックのルール追加。エラーメッセージの文言調整。 ・JUnitを用いた単体テスト補助。テストデータ作成や、テスト実行後のログ確認。 ・障害票の再現手順を作成し、先輩エンジニアが修正した内容を検証。修正後、関連する機能の回帰テストも実施。 ・SubversionからGitへの移行作業補助。既存のソースコードを整理し、.gitignoreファイルの作成や、ブランチ戦略の導入を支援。 【課題・工夫】 エンタープライズ向けシステムのため、業務ロジックが非常に複雑で、初見では理解が困難だった。先輩エンジニアにコードの解説を仰ぎながら、自分なりにシーケンス図を描いて理解を深める習慣をつけた。テストコードにはコメントを丁寧に記述し、テストの意図が後から見ても分かるように心がけた。障害票の再現テストでは、単に「再現する/しない」だけでなく、どの環境で発生するか、どの手順で発生するかを詳細に記録することで、先輩エンジニアのデバッグ効率向上に貢献した。 【実績】 テスト補助と手順書更新により、帳票まわりの回帰確認項目を約25件整理し、繰り返し確認作業の一部効率化に貢献。インターン期間を通じて基本的な開発プロセス、レビュー対応、報告・相談の進め方を習得した。	【言語】 Java 8, Python 3.5 【FW】 Spring, JUnit 【DB】 MySQL 【その他】 Git, Subversion	【規模】 4名 【役割】 インターン / 開発補助 【従事割合】 約40%
【退職理由】 Web制作・受託開発で身につけた実装力を自社サービスの継続改善に活かしたいと考え退職。		

■テクニカルスキル

種類	スキル	使用期間	レベル
OS	Linux	約6年	AWS / GCP / Azure / Docker環境でのWebサービス開発・運用、ログ調査、基本的なサーバー操作が可能
	Windows / macOS	業務全期間	ローカル開発、検証、ドキュメント作成で日常的に利用
言語	TypeScript / JavaScript	約8年	主軸。React / Next.js / Node.js / NestJSで設計・実装・レビュー・若手支援が可能
	PHP	約7年	準主軸。LaravelでAPI・管理画面・運用機能の開発が可能
	SQL	約9年	主軸。MySQL系（RDS, Aurora）の検索・集計・性能改善、インデックス検討が可能
	Python	約2年	補助領域。検証スクリプト、AI API連携、データ移行バッチ、業務補助ツール作成で利用
FW	React / Next.js	約5年	最も得意。SPA開発、画面設計、コンポーネント設計、SSR/CSRを意識した実装が可能
	Vue.js / Nuxt.js	約4年	実務経験あり。管理画面、既存機能改善、状態管理を含むSPA開発が可能
	Node.js / NestJS	約3年	主軸。API実装（REST/GraphQL）、Webhook連携、バッチ処理、外部サービス連携が可能
	Laravel	約7年	準主軸。API、管理画面、認証、通知、運用機能の開発が可能
クラウド・インフラ	AWS	約4年	主軸。CloudFront, ECS, EC2, RDS, S3, WAF, Batch, Lambda, SQSの利用経験あり
	GCP / Azure	約1年	実務経験あり。GCP（BigQuery, Cloud Run）、Azure（基本的なVM, ネットワーク設定）の利用経験
	Terraform	約1年	実務経験あり。既存IaCの改修・レビュー・環境差分確認、新規環境構築が中心
	CI/CD	約6年	主軸。GitHub Actions, AWS CodePipeline, CircleCIによるテスト・デプロイ改善が可能
その他	AI活用 / AI API連携	約2年	実務経験あり。開発支援ツール導入、OpenAI API連携、RAG構築、AI出力レビュー設計が可能
	Docker / Kubernetes	約7年	ローカル開発環境、検証環境、ECS/EKSでのコンテナオーケストレーション利用経験あり
	監視・運用	約2年	Sentry, CloudWatch, Prometheus, Grafanaによるエラー調査、通知整理、改善対応が可能
	コラボレーション	約5年	Slack, GitHub, Backlog, Google Meet, Zoomでのリモート開発経験あり

※上記は自己評価に基づくものです

■得意領域・補助領域

最も得意な領域：TypeScript / React / Next.js を用いたSPA開発、Node.js / Laravel を用いたAPI開発、Aurora MySQL / MySQLを前提とした検索・一覧・CSV出力まわりの性能改善。設計・実装・レビュー・若手メンバーへの説明まで対応可能。

実務で継続利用している領域：AWS、GCP（BigQuery）、Terraform、GitHub Actions、Sentry、CloudWatchを用いたWebサービス運用改善。インフラ専任ではないが、アプリケーション開発者として必要な範囲で設定確認・差分レビュー・リリース手順整備・障害調査を行ってきた。

応用領域：生成AI関連は機械学習モデルそのものではなく、既存AI APIや開発支援ツールをWebサービス開発へ組み込む応用エンジニアリングとして経験。また、マルチテナントSaaSのアーキテクチャ設計や、高セキュリティが要求されるシステムのバックエンド設計も強み。

■技術経験補足（簡略）

【フロントエンド（SPA）】

React / Next.js / Vue.js 中心。エラー状態・空状態・ローディングなどユーザーが迷いやすい状態表示を重視。Atomic Designでのコンポーネント分割経験あり。

【バックエンド / API（高負荷・高セキュリティ）】

Laravel / Node.js (NestJS) でのAPI開発、Webhook、バッチ処理。権限不足・外部エラー・二重送信を考慮した設計。マルチテナントSaaSのアーキテクチャ設計経験あり。

【クラウド / CI/CD / IaC】

AWS (CloudFront, ECS, RDS, S3, WAF, SQS) + GCP (BigQuery) + Azure + Terraform / GitHub Actions。リリース手順・ロールバックを文書化し、デプロイの安定化を図る。

【DB / 性能改善】

Aurora MySQL / MySQL のSQLチューニング、インデックス設計、Redisキャッシュを組合せた性能改善。大規模データ（10万件超）の非同期処理設計経験あり。

【監視 / 運用改善】

Sentry / CloudWatch / Prometheus / Grafana でのエラー調査と通知整理。問い合わせを開発課題に変換し改善。

【AI活用 / AI API連携】

OpenAI APIを用いたFAQ検索・回答候補提示（RAG）、および開発支援AI（Claude, Copilot）のワークフロー導入。AI出力は人が最終判断。

■資格

2018年08月	基本情報技術者試験 合格
2021年11月	AWS Certified Solutions Architect – Associate
2023年07月	AWS Certified Developer – Associate
2025年04月	HashiCorp Certified: Terraform Associate

※英語での技術ドキュメントの読み書き・コミュニケーションにも対応可能。

■自己PR（失敗と学び）

【CSV出力タイムアウトの教訓】

キーワード調査SaaSでCSV出力を同期的に実装したところ、大量データでタイムアウトが多発しユーザーから「出力できない」という問い合わせが殺到した。この失敗から「仕様通り動くこと」と「実際に使いやすいこと」は別だと痛感し、以降は非同期処理・進捗表示・再試行導線をセットで設計する習慣が身についた。現在は実装前にCSや運用担当者へのヒアリングを必ず行い、リリース後も問い合わせログをモニタリングして改善を継続するサイクルを回している。

【リモートチームへの貢献】

フルリモート環境では、GitHub PRやBacklogチケットで「なぜその実装を選んだか」「確認してほしいポイント」を明記するテンプレートをチームに展開し、レビューコストを約20%削減した経験がある。また、ADR（Architecture Decision Record）を活用した技術的意思決定のドキュメント化を推進し、非同期コミュニケーションでも認識齟齬が起きにくい体制を整えた。

【今後取り組みたいこと】

Acompanyの「Trust. Data. AI.」というミッションや「Be Cool, Be Hacker」というバリューに共感する。特に、秘密計算と生成AIを融合した「Confidential AI Suite」のような、技術的難易度が高く社会的インパクトの大きい領域で、バックエンドアーキテクチャの設計から運用改善までをリードしたいと考えている。これまでの経験で培った、高負荷・高セキュリティなシステムの設計ノウハウや、事業成長と技術的負債のバランスを考慮した意思決定力を、Acompanyの急成長フェーズに貢献できると確信している。

以上